

Unidad 4: Replicación y particionamiento

Clase 9 – Persistencia de objetos e integración con aplicaciones

Objetivo

Comprender los principios de la persistencia de objetos y su integración con bases de datos NoSQL, analizando modelos de replicación y consistencia desde la perspectiva de las aplicaciones.

Tema

Modelos de replicación: Master–Slave, Peer to Peer, Quorum. Tipos de consistencia: Eventual, Plena de lectura / escritura. Bases de datos orientadas a objetos (conceptual). Persistencia de objetos (introducción).

Persistencia de objetos e integración con aplicaciones

Del objeto en memoria al dato persistente

Toda aplicación que trabaja con información necesita representar datos mientras se ejecuta. En los lenguajes orientados a objetos, esa representación suele adoptar la forma de objetos. Un objeto puede entenderse como una unidad de software que agrupa datos y comportamiento. Por ejemplo, un sistema de gestión de turnos puede tener objetos como paciente, médico, turno, agenda, especialidad o consulta. Cada uno de esos objetos posee atributos que describen su estado y métodos que definen acciones posibles.

Un objeto Turno, por ejemplo, podría contener una fecha, una hora, un paciente, un médico, un estado y una especialidad. Además, podría incluir comportamientos como reservar, cancelar, confirmar asistencia o marcar como atendido. Mientras la aplicación está funcionando, ese objeto existe en memoria. El problema es que la memoria es temporal: si el programa se cierra, si el servidor se reinicia o si ocurre una falla, los objetos desaparecen.

La persistencia surge para resolver ese problema. Persistir significa guardar el estado relevante de un objeto en un medio durable, de modo que la información pueda recuperarse más adelante. Ese medio puede ser una base de datos relacional, una base documental, una base clave–valor, una base orientada a columnas, un grafo, un archivo o una combinación de varios sistemas de almacenamiento.

La persistencia no consiste simplemente en “guardar algo”. Es un proceso de transformación. El objeto existe en el mundo de la aplicación; la base de datos tiene su propio modelo de representación. Por eso, al persistir, el estado del objeto debe adaptarse al formato y a las reglas del sistema de almacenamiento elegido.

Un mismo objeto de aplicación puede persistirse de distintas formas según el modelo de datos. Un pedido de e-commerce podría guardarse como una fila en una base relacional, como un documento en MongoDB, como una serie de eventos en Cassandra, como una clave temporal en Redis o como parte de una red de relaciones en una base de grafos. La elección no depende solamente de la forma del objeto, sino del uso que la aplicación necesita hacer de esa información.

Estado y comportamiento del objeto

Un objeto combina estado y comportamiento. El estado está dado por los valores de sus atributos en un momento determinado. El comportamiento está definido por sus métodos, es decir, por las operaciones que el objeto puede realizar o responder.

Por ejemplo, un objeto CuentaBancaria puede tener como estado el número de cuenta, el saldo, el titular y el estado de la cuenta. Su comportamiento puede incluir depositar, extraer, bloquear, habilitar o consultar saldo. Cuando se persiste una cuenta bancaria, normalmente se guarda su estado: número, titular, saldo y estado. No se guardan los métodos como tales, porque los métodos pertenecen al código de la aplicación.

Esta diferencia es importante. Persistir un objeto no significa guardar el objeto completo en el mismo sentido en que existe en memoria. En la mayoría de los sistemas, se guarda una representación de sus datos. Luego, cuando la aplicación necesita reconstruirlo, recupera esos datos y vuelve a crear una instancia del objeto con el estado persistido.

En términos prácticos, la persistencia permite que una aplicación pueda cerrar, reiniciarse o ejecutarse en otro servidor sin perder la información fundamental del negocio. Sin persistencia, un sistema de ventas perdería sus pedidos al apagarse; un banco perdería sus movimientos; una plataforma educativa perdería entregas, calificaciones y usuarios registrados.

Representación del objeto en diferentes modelos de datos

La forma en que un objeto se representa en una base de datos depende del modelo utilizado. Cada modelo tiene una lógica distinta y responde mejor a determinados problemas.

En una base relacional, la información se organiza en tablas, filas y columnas. Un objeto complejo suele descomponerse en varias tablas relacionadas. Por ejemplo, un pedido podría dividirse en una tabla de pedidos, una tabla de clientes, una tabla de productos y una tabla de detalle de pedido. Este enfoque favorece la normalización, la integridad referencial y las consultas estructuradas mediante SQL.

En una base documental, como MongoDB, la información se organiza en documentos, generalmente con una estructura similar a JSON. Un pedido podría almacenarse como un único documento que contiene los datos del cliente, la fecha, el estado y una lista embebida de productos comprados. Este enfoque puede resultar más cercano a la

estructura de ciertos objetos de aplicación, especialmente cuando se trabaja con datos jerárquicos o agregados.

En una base clave–valor, como Redis, la información se accede mediante una clave única. Este modelo es muy útil para estados rápidos, caché, sesiones, bloqueos temporales, contadores o reglas de negocio que requieren baja latencia. Por ejemplo, el estado actual de un turno podría almacenarse temporalmente con una clave como turno:4589:estado, cuyo valor podría ser reservado.

En una base wide-column, como Cassandra, los datos se organizan en tablas diseñadas a partir de las consultas esperadas. Es muy utilizada para grandes volúmenes de eventos, registros temporales, sensores, trazabilidad o logs. Un evento de una aplicación, como “turno reservado”, “turno cancelado” o “pago confirmado”, puede almacenarse eficientemente si se define una clave de partición adecuada y un criterio de ordenamiento temporal.

En una base de grafos, como Neo4j, lo central no es solamente el dato individual, sino la relación entre datos. Este modelo permite representar nodos y relaciones. En un sistema de salud, por ejemplo, podrían analizarse relaciones entre pacientes, médicos, especialidades, centros médicos, turnos, diagnósticos y tratamientos. Este enfoque es especialmente valioso cuando las preguntas de negocio dependen de patrones de conexión.

La conclusión fundamental es que no existe una única forma correcta de persistir un objeto. La representación persistente debe responder a la estructura del dato, al volumen esperado, a los patrones de consulta, a los tiempos de respuesta requeridos y al nivel de consistencia necesario.

Bases de datos orientadas a objetos

Las bases de datos orientadas a objetos son sistemas de almacenamiento diseñados para guardar objetos de manera más directa que los modelos tradicionales. En lugar de transformar los objetos en tablas o documentos, buscan conservar aspectos propios del paradigma orientado a objetos, como identidad, atributos complejos, relaciones entre objetos, colecciones, herencia y encapsulamiento.

La identidad de objeto significa que cada objeto puede reconocerse como una entidad única, independientemente de los valores de sus atributos. Esto se diferencia de un registro relacional clásico, donde la identificación suele depender de una clave primaria. En el mundo orientado a objetos, dos objetos podrían tener atributos iguales, pero seguir siendo objetos distintos.

La herencia permite que una clase especializada derive de otra clase más general. Por ejemplo, EmpleadoAdministrativo, EmpleadoMedico y EmpleadoTecnico podrían heredar de una clase general Empleado. Una base orientada a objetos puede representar este tipo de estructuras de forma más natural que una base relacional, donde la herencia debe transformarse en tablas mediante distintas estrategias de diseño.

El encapsulamiento implica que un objeto protege su estado interno y expone operaciones controladas para modificarlo. Aunque las bases de datos orientadas a objetos intentan acercarse a esta lógica, en la práctica la persistencia suele concentrarse en el estado y no tanto en el comportamiento completo del objeto.

Este tipo de bases tuvo importancia conceptual porque intentó reducir la distancia entre el modelo de programación y el modelo de almacenamiento. Sin embargo, en muchos entornos empresariales actuales predominan las bases relacionales, las bases NoSQL y las arquitecturas híbridas. Por eso, más que pensar solamente en bases puramente orientadas a objetos, resulta útil comprender el problema general de la persistencia de objetos y su adaptación a distintos modelos de almacenamiento.

Persistencia de objetos y bases NoSQL

Las bases NoSQL no deben entenderse como un reemplazo universal de las bases relacionales, sino como una familia de modelos diseñados para resolver problemas donde el modelo tabular tradicional puede resultar limitado, rígido o costoso de escalar. NoSQL significa, en términos amplios, “no solamente SQL”. Incluye bases documentales, clave–valor, wide-column y grafos, entre otras.

La persistencia de objetos en bases NoSQL exige comprender qué parte del objeto se quiere guardar, cómo será consultada y qué nivel de transformación se necesita. En algunos casos, el objeto puede tener una representación bastante directa. En otros, conviene descomponerlo, duplicar información o diseñar estructuras específicas para una consulta.

Una base documental puede ser adecuada cuando el objeto tiene una estructura jerárquica o compuesta. Por ejemplo, un pedido con sus ítems puede persistirse en un documento que incluya datos generales del pedido y un arreglo de productos. Esta representación facilita recuperar el pedido completo en una sola lectura. Sin embargo, puede generar duplicación de datos si se embebe información que también existe en otros documentos, como datos del cliente o del producto.

Una base clave–valor puede ser adecuada cuando la aplicación necesita acceder rápidamente a un dato puntual. Por ejemplo, saber si un usuario está bloqueado, cuántos intentos fallidos acumula o cuál es el token de sesión activo. En estos casos, el dato persistido no busca representar toda la riqueza del objeto, sino un estado operativo específico.

Una base wide-column puede ser adecuada cuando la aplicación genera grandes cantidades de eventos. Por ejemplo, cada vez que un usuario inicia sesión, reserva un turno, cancela una operación o modifica su perfil, se puede registrar un evento. Estos eventos pueden consultarse por usuario, por fecha, por tipo de acción o por entidad afectada. El diseño se orienta a las consultas concretas, no a la normalización.

Una base de grafos puede ser adecuada cuando el análisis depende de relaciones. Por ejemplo, detectar pacientes que comparten médicos, dispositivos, direcciones, cuentas

o patrones de uso. En un caso de fraude, las conexiones pueden ser más importantes que los datos aislados.

La persistencia en NoSQL obliga a pensar desde el uso. No se diseña primero una estructura ideal y luego se consulta de cualquier forma. Muchas veces se parte de las preguntas que la aplicación debe responder y se modela la información para responderlas con eficiencia.

Integración entre aplicaciones y bases de datos

La integración entre una aplicación y una base de datos define cómo se comunican ambos mundos. La aplicación administra objetos, reglas de negocio, interfaces de usuario, servicios y procesos. La base de datos administra almacenamiento, recuperación, replicación, índices, disponibilidad y consistencia.

Integrar no significa únicamente abrir una conexión y ejecutar una operación de guardado. También implica definir responsabilidades. La aplicación debe saber cuándo crear un objeto, cuándo validarlo, cuándo transformarlo, cuándo enviarlo a la base, cómo manejar errores y cómo responder si la operación no se completa.

Por ejemplo, si un usuario intenta reservar un turno médico, la aplicación debe verificar si el turno existe, si está disponible, si el paciente está habilitado, si el médico atiende esa especialidad y si no hay conflicto de horarios. Luego debe persistir la reserva. Si la escritura falla, debe decidir si informa el error, reintenta, deja la operación pendiente o registra un evento compensatorio.

En sistemas distribuidos, la integración es más compleja porque la base puede tener múltiples nodos, réplicas o servicios asociados. La aplicación puede escribir en un nodo y leer desde otro. Puede recibir una confirmación parcial. Puede encontrar datos temporalmente desactualizados. Puede perder conexión con una réplica. Puede haber demoras entre la escritura y la propagación del cambio.

Por eso, la integración debe diseñarse considerando no solo el modelo de datos, sino también el comportamiento esperado ante fallas. Una aplicación robusta no asume que todo guardado será inmediato, que toda lectura será perfecta o que toda réplica estará actualizada. Diseña mecanismos para manejar esas situaciones.

Mapeo entre objetos y datos persistentes

El mapeo es el proceso mediante el cual se transforma un objeto de aplicación en una estructura persistente y viceversa. En una base relacional, este proceso suele conocerse como mapeo objeto-relacional. En otros modelos puede hablarse de mapeo objeto-documento, objeto-clave, objeto-evento u objeto-grafo, aunque estos términos no siempre se usen formalmente.

El mapeo implica decidir qué atributos se guardan, con qué nombres, en qué estructura, con qué identificadores, con qué relaciones y con qué nivel de duplicación. También implica decidir cómo se reconstruye el objeto cuando se recupera desde la base.

Por ejemplo, un objeto Paciente puede tener atributos como nombre, documento, correo, teléfono, cobertura médica y domicilio. En una base documental, esos datos podrían guardarse juntos en un documento. En una base relacional, podrían separarse en varias tablas. En Redis, quizá solo se guarde temporalmente el estado de sesión del paciente. En un grafo, el paciente podría ser un nodo conectado con médicos, turnos, instituciones y tratamientos.

El mapeo debe evitar dos errores frecuentes. El primero es querer guardar el objeto exactamente como está en memoria, sin considerar cómo se va a consultar. El segundo es diseñar la base sin considerar cómo trabaja la aplicación. Una buena integración requiere equilibrio: el modelo persistente debe ser eficiente para la base, pero también comprensible y utilizable para la aplicación.

Replicación de datos

La replicación es el proceso mediante el cual se mantienen copias de los datos en más de un nodo. Un nodo puede entenderse como una instancia o servidor que participa en el almacenamiento y procesamiento de datos dentro de un sistema distribuido.

Replicar datos permite mejorar la disponibilidad, aumentar la tolerancia a fallos, distribuir la carga de lectura y facilitar la recuperación ante incidentes. Si una base de datos tiene una sola copia y ese servidor falla, el sistema puede quedar inoperativo. En cambio, si existen réplicas, otro nodo puede responder o ayudar a recuperar el servicio.

La replicación también puede mejorar el rendimiento. Si muchas aplicaciones o usuarios consultan los mismos datos, las lecturas pueden distribuirse entre distintas réplicas. Esto reduce la presión sobre un único servidor y mejora la capacidad de respuesta.

Sin embargo, replicar introduce una dificultad técnica importante: mantener la coherencia entre copias. Si un dato se modifica en un nodo, las demás réplicas deben actualizarse. La actualización puede ser inmediata o diferida. Puede requerir confirmación de todos los nodos o solo de algunos. Puede aceptar demoras o exigir sincronización estricta.

Por ejemplo, si un paciente cambia su número de teléfono y el dato está replicado en tres nodos, el sistema debe decidir cuándo se considera válido el cambio. ¿Alcanza con que lo registre un nodo? ¿Deben confirmarlo dos? ¿Deben confirmarlo los tres? ¿Qué ocurre si un nodo está temporalmente desconectado? Estas preguntas muestran que la replicación no puede analizarse separada de la consistencia.

Modelo Master–Slave

El modelo Master–Slave, también conocido como primario-secundario, organiza la replicación a partir de un nodo principal y uno o más nodos secundarios. El nodo master, o primario, recibe las operaciones principales de escritura. Los nodos slave, o secundarios, reciben copias de los datos desde el nodo principal.

Este modelo es relativamente simple de comprender. La aplicación escribe en el nodo principal, y ese nodo propaga los cambios hacia las réplicas. Las lecturas pueden realizarse desde el nodo principal o desde los secundarios, según la configuración y el nivel de consistencia requerido.

La principal ventaja de este modelo es que centraliza el control de las escrituras. Al haber un nodo principal, se reduce el conflicto entre múltiples escrituras simultáneas en distintos lugares. Además, permite usar réplicas secundarias para consultas de lectura, reportes, análisis o respaldo.

Un ejemplo práctico sería una aplicación transaccional que registra pedidos en una base principal, mientras que los reportes de ventas se ejecutan sobre réplicas secundarias. De esta manera, las consultas analíticas no sobrecargan el nodo que procesa las operaciones del negocio.

El principal riesgo del modelo es que el nodo master puede convertirse en un punto crítico. Si falla, el sistema necesita un mecanismo para promover un nodo secundario a principal. Este proceso se conoce como failover, que significa conmutación por error. Si el failover no está bien diseñado, puede haber interrupciones, pérdida de escrituras recientes o conflictos.

Otro riesgo aparece cuando la replicación es asíncrona. Asíncrona significa que el nodo principal confirma la operación sin esperar necesariamente a que todas las réplicas estén actualizadas. Esto mejora el rendimiento, pero puede permitir que una lectura desde un nodo secundario devuelva información vieja durante un breve período.

Modelo Peer to Peer

El modelo Peer to Peer, o entre pares, elimina la idea de un único nodo principal. En este esquema, los nodos tienen roles similares y pueden participar en la recepción, almacenamiento y distribución de datos. La arquitectura busca evitar un centro único de control y distribuir la carga entre varios nodos.

Este modelo es frecuente en sistemas distribuidos que priorizan escalabilidad horizontal y alta disponibilidad. Escalabilidad horizontal significa aumentar la capacidad del sistema agregando más nodos, en lugar de depender únicamente de un servidor más potente.

En una arquitectura Peer to Peer, una aplicación puede conectarse a distintos nodos del cluster. Un cluster es un conjunto de nodos que trabajan coordinadamente como parte de un mismo sistema. Cada nodo puede encargarse de una parte de los datos o colaborar en la replicación de la información.

La ventaja principal es que no existe un único punto de fallo tan marcado como en un esquema primario-secundario. Si un nodo deja de responder, otros nodos pueden seguir funcionando. Además, la carga puede distribuirse mejor, lo que resulta útil para sistemas con grandes volúmenes de escritura o lectura.

El desafío está en la coordinación. Si varios nodos pueden aceptar operaciones, el sistema necesita mecanismos para saber qué versión de un dato es válida, cómo resolver conflictos, cómo propagar cambios y cómo recuperar la consistencia cuando un nodo vuelve después de una falla.

Un ejemplo aplicado puede encontrarse en sistemas de eventos masivos. Una plataforma que recibe millones de registros de actividad por hora necesita distribuir escritura y almacenamiento. En ese contexto, una arquitectura entre pares permite repartir el trabajo y sostener disponibilidad aun cuando algunos nodos fallen.

Modelo Quorum

El modelo Quorum se basa en exigir que una cantidad mínima de nodos confirme una operación para considerarla válida. No siempre se requiere que respondan todos los nodos, pero tampoco alcanza necesariamente con uno solo. El sistema define un umbral mínimo de confirmaciones.

La palabra quorum hace referencia a una cantidad suficiente de participantes para validar una decisión. En bases distribuidas, se usa para equilibrar consistencia, disponibilidad y rendimiento.

Por ejemplo, si un dato está replicado en tres nodos, la aplicación podría requerir que al menos dos nodos confirmen una escritura. De ese modo, aunque un nodo falle o esté demorado, la operación puede completarse con un nivel razonable de seguridad.

El quorum puede aplicarse a lecturas y escrituras. Un quorum de escritura define cuántas réplicas deben confirmar que guardaron el dato. Un quorum de lectura define cuántas réplicas deben consultarse para obtener una respuesta confiable.

La combinación entre quorum de lectura y quorum de escritura permite ajustar el nivel de consistencia. Si se exige mucho quorum, aumenta la seguridad de que el dato leído sea el correcto, pero también aumenta la latencia. Latencia significa demora entre la solicitud de una operación y la respuesta del sistema.

Si se exige poco quorum, el sistema responde más rápido y tolera mejor fallas, pero aumenta el riesgo de leer datos desactualizados. Por eso, quorum no es una solución única, sino un mecanismo configurable según la criticidad del dato.

Un sistema de recomendaciones puede aceptar respuestas rápidas con menor consistencia. Si una recomendación no considera la última acción del usuario, el impacto puede ser bajo. En cambio, una operación de reserva, stock o pago puede requerir mayor confirmación antes de responder como exitosa.

Consistencia de datos

La consistencia se refiere al grado en que los datos se mantienen correctos, actualizados y coherentes entre las distintas partes del sistema. En una base distribuida, donde existen múltiples nodos y réplicas, la consistencia se vuelve un problema central.

La pregunta práctica es simple: cuando una aplicación modifica un dato, ¿todos los usuarios y servicios ven inmediatamente el mismo valor?

Si un usuario cambia su domicilio, un servicio puede leer el dato actualizado desde una réplica, mientras otro servicio podría leer todavía el domicilio anterior desde otra réplica. Esa diferencia puede durar milisegundos, segundos o más tiempo, según el diseño del sistema.

No toda inconsistencia temporal es necesariamente un error. En muchos sistemas distribuidos, se acepta cierto nivel de demora para mejorar disponibilidad y rendimiento. El punto clave es decidir qué datos pueden tolerar esa demora y cuáles no.

La consistencia no debe analizarse de manera abstracta. Debe evaluarse desde el impacto en la aplicación. No es lo mismo mostrar una foto de perfil anterior que confirmar dos veces el mismo turno médico. No es lo mismo demorar una estadística que informar mal un saldo bancario. No es lo mismo actualizar un ranking que vender stock inexistente.

Consistencia eventual

La consistencia eventual significa que, si no se realizan nuevas modificaciones sobre un dato, todas las réplicas terminarán convergiendo hacia el mismo valor. El sistema no garantiza que todos los nodos tengan la última versión inmediatamente después de una escritura, pero sí que alcanzarán la coherencia con el tiempo.

Este modelo es común en sistemas distribuidos que priorizan disponibilidad, escalabilidad y baja latencia. Permite que la aplicación siga funcionando aunque algunas réplicas estén desactualizadas temporalmente.

Un ejemplo claro es una red social. Si un usuario cambia su foto de perfil, puede ocurrir que algunos contactos vean la foto nueva y otros vean la anterior durante unos segundos. En la mayoría de los casos, esa demora es aceptable. Lo importante es que, después de un tiempo, todas las réplicas muestren la versión actualizada.

Otro ejemplo puede ser un sistema de recomendaciones. Si un usuario acaba de ver una película y el motor de recomendaciones todavía no incorporó ese evento en todos los nodos, el impacto no suele ser crítico. La recomendación puede mejorar unos segundos o minutos más tarde.

La consistencia eventual no significa ausencia de control. Significa aceptar que la sincronización no es inmediata. El sistema debe estar diseñado para manejar esa realidad. La aplicación no debería usar consistencia eventual para operaciones donde una lectura vieja pueda generar pérdidas, errores legales, conflictos operativos o decisiones irreversibles.

Consistencia plena de lectura

La consistencia plena de lectura implica que cuando la aplicación consulta un dato, debe obtener la versión más actualizada y válida según las reglas del sistema. En términos prácticos, la aplicación no debería leer información vieja después de que una actualización fue confirmada.

Este nivel de consistencia es importante cuando una lectura incorrecta puede afectar una operación posterior. Por ejemplo, si una aplicación muestra turnos disponibles, la lectura debe ser suficientemente confiable para evitar que dos pacientes intenten reservar el mismo horario. Si el sistema lee desde una réplica desactualizada, podría mostrar como libre un turno que ya fue tomado.

También es importante en sistemas de stock. Si una tienda online muestra unidades disponibles basándose en una lectura vieja, puede aceptar compras que luego no podrá cumplir. En ese caso, el problema no es solo técnico; también afecta la experiencia del cliente y la operación del negocio.

Lograr consistencia plena de lectura puede requerir consultar al nodo principal, usar quorum de lectura, bloquear ciertos datos, invalidar caché o coordinar la lectura con escrituras recientes. Estas estrategias suelen tener un costo en latencia o disponibilidad, pero pueden ser necesarias para datos críticos.

Consistencia plena de escritura

La consistencia plena de escritura implica que una operación de escritura se considera válida solamente cuando el sistema garantiza que fue registrada de forma segura según el nivel de confirmación definido. La aplicación no debería informar éxito si la operación todavía puede perderse o quedar en un estado incierto.

Este tipo de consistencia es fundamental en operaciones críticas. Una transferencia bancaria, una reserva de turno, una compra de stock limitado, una modificación de permisos o una confirmación de pago no deberían depender de una escritura débil o no confirmada.

Por ejemplo, si dos usuarios intentan comprar la última unidad de un producto, la escritura que descuenta el stock debe estar controlada. No alcanza con que cada aplicación lea "stock disponible" desde una réplica distinta y luego confirme ambas ventas. El sistema debe garantizar que la escritura final respete la regla de negocio: no vender más unidades que las existentes.

La consistencia plena de escritura puede requerir transacciones, bloqueo lógico, confirmación por quorum, escritura en nodo principal, control de versión o mecanismos de concurrencia. La concurrencia aparece cuando varias operaciones intentan acceder o modificar los mismos datos al mismo tiempo.

El beneficio de una escritura consistente es la confiabilidad. El costo suele ser mayor demora, menor flexibilidad ante fallas o menor disponibilidad temporal si no se logra la confirmación requerida.

Relación entre replicación, disponibilidad y consistencia

La replicación busca mejorar la disponibilidad y la tolerancia a fallos, pero al mismo tiempo introduce desafíos de consistencia. Cuantas más copias existen, más coordinación se necesita para garantizar que todas reflejen el mismo estado.

La disponibilidad indica la capacidad del sistema para responder cuando se lo necesita. Un sistema altamente disponible intenta seguir funcionando aun cuando algunos componentes fallen. Sin embargo, para mantener esa disponibilidad, a veces se acepta que ciertas réplicas respondan con datos que todavía no están completamente actualizados.

La consistencia indica el grado de coherencia y actualización de los datos. Para lograr consistencia fuerte, el sistema puede necesitar esperar confirmaciones, coordinar nodos o rechazar operaciones si no puede garantizar el estado correcto. Esto puede reducir la disponibilidad en algunos escenarios.

La tensión entre disponibilidad y consistencia es una característica central de los sistemas distribuidos. No se trata de elegir siempre una sobre la otra, sino de definir qué prioridad corresponde a cada tipo de dato y a cada operación.

Un sistema de salud puede aceptar consistencia eventual para estadísticas agregadas de demanda, pero no para confirmar un turno quirúrgico. Una plataforma de streaming puede aceptar consistencia eventual en recomendaciones, pero necesita mayor consistencia en facturación. Un sistema financiero puede distribuir reportes con cierta demora, pero no puede tratar del mismo modo la actualización de saldos.

La arquitectura correcta no aplica el mismo criterio a todo. Clasifica datos y operaciones según criticidad, impacto, frecuencia, volumen y tolerancia al error.

Persistencia políglota

La persistencia políglota es un enfoque arquitectónico que utiliza más de un modelo de base de datos dentro de una misma solución. La palabra políglota remite a la capacidad de hablar varios lenguajes. En este contexto, significa usar distintos “lenguajes de almacenamiento” según el problema.

Una aplicación moderna puede usar una base relacional para datos maestros, MongoDB para documentos flexibles, Redis para caché y estados temporales, Cassandra para eventos masivos y Neo4j para análisis de relaciones. No se trata de acumular tecnologías, sino de asignar cada tipo de dato al modelo que mejor resuelve su uso.

Por ejemplo, en una aplicación de logística, los datos de clientes y facturación pueden mantenerse en una base relacional. Los eventos de seguimiento de paquetes pueden almacenarse en Cassandra por volumen y orden temporal. El estado actual de una entrega puede exponerse desde Redis para lectura rápida. Las relaciones entre paquetes, rutas, depósitos, transportistas y eventos anómalos pueden analizarse en un grafo.

El beneficio de la persistencia políglota es que permite optimizar cada componente. El riesgo es aumentar la complejidad. Más bases implican más integraciones, más monitoreo, más reglas de consistencia, más seguridad, más mantenimiento y más conocimiento técnico requerido.

Por eso, la persistencia políglota debe justificarse. No conviene usar varias bases solo por modernidad. Conviene hacerlo cuando existe una razón clara: volumen, velocidad, flexibilidad, relaciones complejas, baja latencia, análisis temporal o escalabilidad.

Criterios para decidir el modelo de persistencia

La selección de un modelo de persistencia debe partir de preguntas concretas. La primera es qué tipo de dato se desea almacenar. No es lo mismo un dato maestro, una transacción, un evento, una sesión, una relación o una métrica agregada.

La segunda pregunta es cómo se va a consultar. En bases NoSQL, especialmente en modelos documentales y wide-column, el patrón de consulta influye directamente en el diseño. Si la aplicación necesita recuperar un pedido completo con sus ítems, un documento embebido puede ser conveniente. Si necesita consultar millones de eventos por usuario y fecha, una estructura orientada a partición temporal puede ser mejor.

La tercera pregunta es qué volumen y velocidad se esperan. Algunos modelos funcionan bien con datos moderados y consultas complejas; otros están diseñados para escrituras masivas, baja latencia o escalabilidad horizontal.

La cuarta pregunta es qué nivel de consistencia requiere cada operación. Si una lectura vieja no genera daño, puede aceptarse consistencia eventual. Si una escritura incorrecta puede generar pérdida económica, doble reserva, error legal o inconsistencia operativa, se requiere mayor control.

La quinta pregunta es qué ocurre ante fallas. Una arquitectura debe prever nodos caídos, demoras de red, réplicas desactualizadas, escrituras parciales, timeouts y reintentos. Timeout significa que una operación supera el tiempo máximo de espera definido por el sistema.

La sexta pregunta es quién será responsable de cada dato. En arquitecturas con varios servicios, es importante definir qué componente tiene autoridad para escribir o modificar cada información. Si muchos servicios modifican el mismo dato sin coordinación, aumentan los conflictos.

La séptima pregunta es cómo se auditan los cambios. En sistemas empresariales, no alcanza con conocer el estado actual. Muchas veces se necesita saber qué ocurrió, cuándo ocurrió, quién lo hizo, desde dónde y con qué resultado. Por eso, los eventos de auditoría pueden tener un modelo de persistencia propio.

Ejemplo aplicado: sistema de reservas médicas

Un sistema de reservas médicas permite visualizar profesionales, consultar disponibilidad, reservar turnos, cancelar citas, enviar recordatorios y generar estadísticas de demanda. Aunque parezca una única aplicación, internamente maneja datos con necesidades muy diferentes.

Los datos de pacientes y médicos son datos relativamente estables. Pueden requerir integridad, validaciones y protección de información sensible. Podrían persistirse en una base relacional o documental, según el diseño general del sistema.

La agenda médica requiere mayor control. Si un turno está disponible, la aplicación debe evitar que dos pacientes lo reserven al mismo tiempo. Esto exige consistencia fuerte en la escritura o mecanismos equivalentes de control de concurrencia.

Los recordatorios pueden manejarse como eventos o mensajes programados. Si un recordatorio se envía unos segundos más tarde, normalmente el impacto es bajo. En este caso puede aceptarse una arquitectura más flexible, siempre que exista trazabilidad.

Las estadísticas de demanda pueden aceptar consistencia eventual. Si el reporte de turnos por especialidad se actualiza con algunos minutos de demora, probablemente no afecte la operación inmediata.

Los eventos de auditoría, como creación, cancelación o modificación de turnos, deben registrarse de forma confiable. Aunque no siempre requieren lectura inmediata, sí requieren durabilidad y trazabilidad.

Este ejemplo muestra que una misma aplicación puede necesitar distintos niveles de persistencia, replicación y consistencia. El error sería tratar todos los datos de la misma manera.

Ejemplo aplicado: e-commerce

Un sistema de e-commerce permite registrar usuarios, mostrar productos, gestionar carritos, confirmar pagos, descontar stock y generar recomendaciones. Cada parte tiene requisitos distintos.

El catálogo de productos puede replicarse para mejorar la velocidad de lectura. Si una descripción tarda unos segundos en actualizarse en todas las réplicas, el impacto puede ser menor.

El carrito de compras puede almacenarse en una base rápida, incluso en Redis, porque necesita baja latencia y puede tener una vida temporal. Sin embargo, al confirmar la compra, el sistema debe pasar a un modelo más confiable.

El pago requiere consistencia fuerte. No debe informarse una compra como aprobada si la operación no quedó confirmada. También se necesita trazabilidad para auditoría, conciliación y reclamos.

El stock requiere especial cuidado. Si el sistema vende más unidades de las disponibles, genera un problema operativo. Por eso, la escritura que descuenta stock debe estar protegida contra concurrencia.

Las recomendaciones pueden aceptar consistencia eventual. Si el sistema no incorpora la última búsqueda del usuario inmediatamente, no se compromete la operación central.

Este tipo de análisis permite diseñar una arquitectura más racional. No todo debe ser rápido a costa de consistencia, ni todo debe ser fuertemente consistente a costa de rendimiento. Cada decisión debe alinearse con el impacto del dato.

Errores frecuentes en el diseño de persistencia

Uno de los errores más comunes es elegir una base de datos por moda tecnológica y no por necesidad. Usar MongoDB, Cassandra, Redis o Neo4j no mejora un sistema si el problema no requiere sus capacidades específicas.

Otro error es diseñar la persistencia copiando directamente el modelo de objetos. Aunque un objeto sea complejo, no siempre conviene guardarlo como una estructura igualmente compleja. La base debe diseñarse pensando en consultas, volumen, consistencia y mantenimiento.

También es frecuente confundir disponibilidad con consistencia. Que un sistema responda rápidamente no significa que responda con el dato correcto. Y que un sistema sea consistente no significa que siempre estará disponible en cualquier escenario de falla.

Otro error es ignorar la actualización de réplicas. Si la aplicación lee desde nodos secundarios sin considerar demoras de replicación, puede tomar decisiones sobre datos viejos.

También puede ser problemático usar caché sin estrategia de invalidación. La caché es una copia rápida de datos usados frecuentemente. Si no se actualiza o invalida correctamente, puede entregar información desactualizada. En datos no críticos puede ser aceptable; en datos críticos puede provocar errores graves.

Un último error relevante es no registrar eventos de auditoría. Muchos sistemas solo guardan el estado final, pero no la historia de cambios. En contextos empresariales, regulatorios o de seguridad, la trazabilidad es fundamental.

Síntesis conceptual

La persistencia de objetos permite transformar el estado de los objetos de una aplicación en datos durables. Esta transformación no es neutra: depende del modelo de base de datos, de las consultas, del volumen, de la velocidad requerida y de la criticidad de la información.

Las bases orientadas a objetos ofrecen una solución conceptual cercana al paradigma de programación, pero en la práctica moderna la persistencia suele resolverse mediante bases relacionales, documentales, clave–valor, wide-column, grafos o combinaciones de ellas.

La integración entre aplicaciones y bases de datos exige definir cómo se guardan, leen, actualizan y validan los datos. En sistemas distribuidos, también exige considerar replicación, fallas, consistencia y disponibilidad.

Los modelos de replicación, como Master–Slave, Peer to Peer y Quorum, ofrecen distintas formas de distribuir copias y confirmar operaciones. Cada uno presenta ventajas y riesgos. No existe un modelo universalmente superior; existe un modelo más o menos adecuado según el problema.

La consistencia define qué tan actualizado y coherente debe estar el dato al leerlo o escribirlo. La consistencia eventual puede ser aceptable para datos de bajo impacto inmediato. La consistencia plena de lectura o escritura es necesaria cuando una decisión incorrecta puede generar conflictos, pérdidas o errores críticos.

Diseñar persistencia en aplicaciones modernas implica clasificar datos, comprender operaciones, anticipar fallas y justificar decisiones. Una arquitectura sólida no es la que usa más tecnologías, sino la que asigna correctamente cada dato al modelo, nivel de replicación y consistencia que necesita.